



Figure 2: Sample page output

- By using the evaluate tag `<% _each( listItems, function( listItem ) { %>`, it iterates the given array of data. The evaluate tag helps to provide the logical structure to the template.

## view.js: An explanation

- Line 1 uses the *jQuery* ready statement, to start the interpretation after loading HTML.
- Line 3 simply defines the template as a string; it is compiled by the statement `_template()` and returns the function. This is called precompilation. This is explained in detail in the next section. This template uses the *interpolate* tag `<%= course %>`. It prints the contents of the course variable.
- In Line 4, data is passed to the function `tpl`. This function replaces the variable with data and it returns the HTML content.
- Line 5 inserts the HTML content to the `<header>` tag by using the *jQuery* function.
- Lines 7-22 define the sample JSON data. In real-time, it might be fetched from the server side API by using an AJAX request.
- Lines 24-26 also work in the same way as Lines 3-5, but it takes the template from the separate script tag by using the *jQuery* selector.

## Template settings

We can use the following settings to change the default behaviour of the template engine:

```
_.templateSettings = {
  evaluate    : <regex>,
  interpolate : <regex>,
  escape     : <regex>,
  variable    : <variable name>
};
```

## Changing the syntax

By default, Underscore uses the ERB style template delimiters. In some cases, there may be a need to change the syntax of the Underscore template to avoid a clash with server-side tags. In the above settings, the first three values

helps to change the syntax of tags in the templates.

You can see the example given below:

```
_.templateSettings = {
  interpolate : /\{\{(.+)\}\}/g,
  evaluate    : /<%(.*?)%>/g,
  escape      : /<%-([^\s%]+)%>/g,
  variable    : "x"
};
var tpl = _.template("<h1>Hello {{x.course}} students,
year {{x.year}}</h1>");
var tplString = tpl({course: "MBA", year:2015});
$("#header").html(tplString);
```

In the above example, Interpolate looks for the tags like `{{ ... }}`. For the evaluate and escape tags, we keep the default Underscore template syntax. If you want to change syntax for a specific tag, keep the default values for the other tags like in the above example. Otherwise, it won't recognise those elements.

## Limiting the scope

The variable parameter helps to limit the scope of the data we pass into the template. Otherwise, the function created by precompilation uses the *with* statement to control the scope. The *with* statement is recognised as a source of confusing bugs and compatibility issues. It is important to set up this *variable* while configuring, to avoid the usage of the *with* statement. It makes the template faster.

We can also set it up as follows:

```
_.templateSettings.variable = "x";
```

## Precompilation

One of the important concepts of the template engine is precompilation. Basically, all template engines use some regular expressions to replace the tags with data. Since regular expressions are very costly in terms of processing, the templates are converted to functions and these functions can be used many times with less processing power. This is called precompilation.

We can see this precompiled template (function) by using *Firebug* (see Figure 4).

The following function is created by using Underscore for the template 'Hello MBA students'.

```
var __t,
    __p='',
    __j=Array.prototype.join,
    print=function(){
      __p+=__j.call(arguments, '');
    };
with(obj){|(){
  __p+='\<h1>Hello ' +
```